

JAVA MODERNE

écrire un code plus robuste et concis

BUT2 Informatique · R3.04 Design patterns · IUT du Limousin

SOMMAIRE

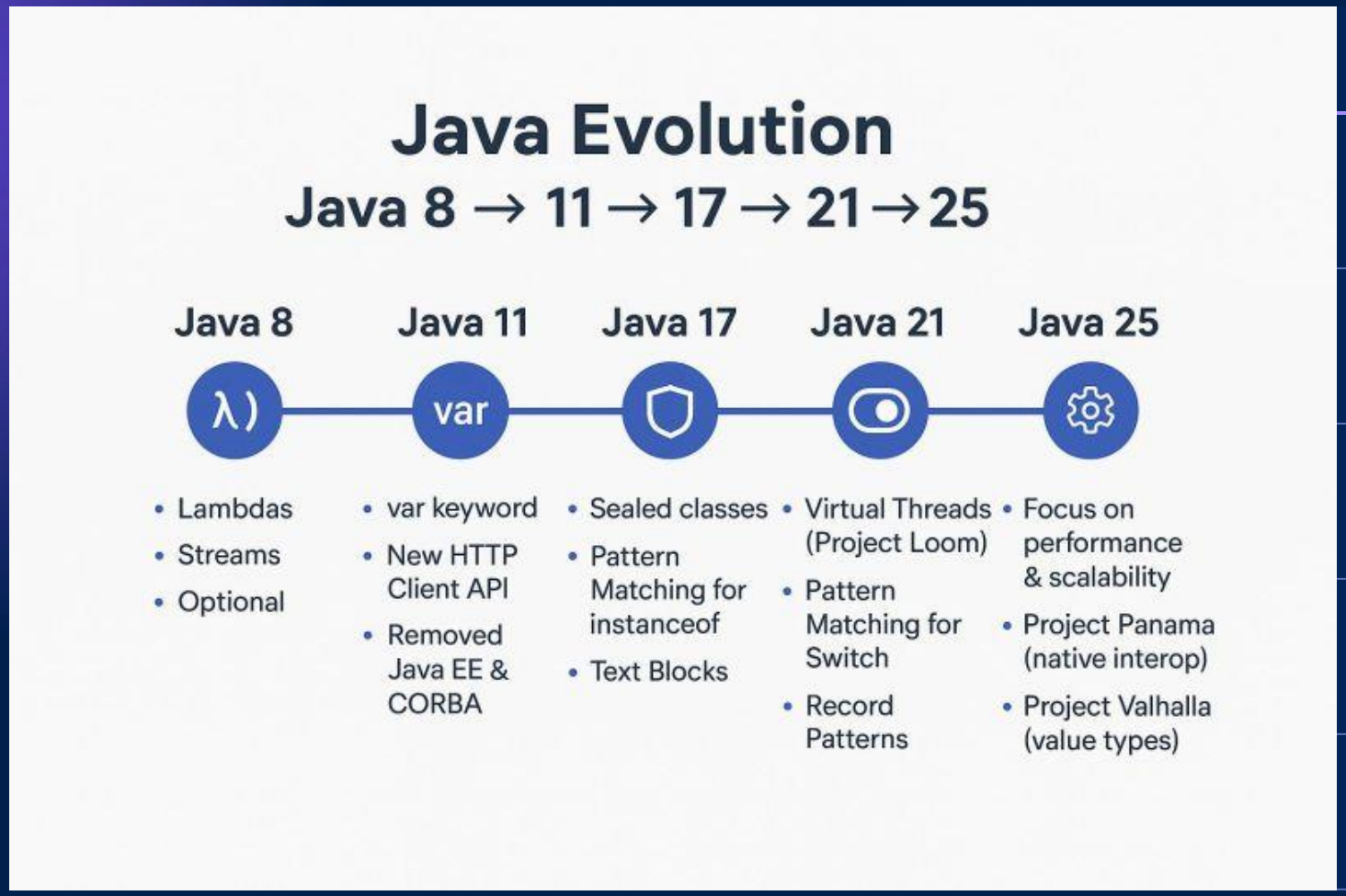


- Pourquoi Java évolue
- Les classes scellées
- Le pattern matching
- Streams et style fonctionnel
- Live-coding
- QCM interactif
- Bibliographie

POURQUOI JAVA ÉVOLUE-T-IL ?

- **Un langage réputé verbeux :** getters, setters, casts explicites, classes anonymes
- **Un rythme accéléré :** une version tous les 6 mois depuis 2017, contre environ 3 ans avant
- **Concurrence :** 
- **Objectif :** un code plus sûr (erreurs détectées à la compilation, pas à l'exécution) et plus concis

FRISE DES VERSIONS : DE JAVA 8 À JAVA 21



Pourquoi
Tirer parti des multicœurs, alléger les classes anonymes
Livrer plus vite, rester compétitif face à Kotlin et Scala
Supprimer le code répétitif : getters, equals, casts
Fermer une hiérarchie : le compilateur connaît tous les cas
Exploiter la hiérarchie fermée : exhaustivité vérifiée

01

LES CLASSES ET INTERFACES SCELLÉES



LE PROBLÈME : UNE INTERFACE TROP OUVERTE


```java
public interface MoyenPaiement {
}
public class CarteBancaire implements MoyenPaiement { .... }

public class Virement implements MoyenPaiement { .... }

public class Especes implements MoyenPaiement { .... }
```

Six mois plus tard, ailleurs dans le projet…

```java
public class Bitcoin
        implements MoyenPaiement { ... }
```

…et un traitement qui oublie un cas

```java
if (m instanceof CarteBancaire) { ... }
else if (m instanceof Virement) { ... }
// Especes oublié : ça compile quand même
```

Rien n'oblige à traiter tous les cas. L'oubli compile quand même : le bug n'apparaît qu'à l'exécution / en production.

LA SOLUTION : LE MOT-CLÉ SEALED


```
public sealed interface MoyenPaiement
        permits CarteBancaire, Virement, Especes { }

public final class CarteBancaire
        implements MoyenPaiement { ... }
public final class Virement
        implements MoyenPaiement { ... }
public final class Especes
        implements MoyenPaiement { }
```

Même module (ou package) : tous les sous-types permis doivent être au même endroit. S'ils sont dans le même fichier, permits peut être omis.

Chaque sous-type choisit son avenir :

- **`final`** : ne peut plus être étendu
- **`sealed`** : autorise une sous-hiérarchie restreinte
- **`non-sealed`** : rouvre volontairement l'héritage

```
public class Bitcoin
    implements MoyenPaiement { ... }
```

Erreur de compilation : Bitcoin n'est pas autorisé dans la liste des permits

SEALED + RECORD :


```
public sealed interface MoyenPaiement
        permits CarteBancaire, Virement, Especes { }

public record CarteBancaire(String numero, String titulaire)
        implements MoyenPaiement { }
public record Virement(String iban)
        implements MoyenPaiement { }
public record Especes()
        implements MoyenPaiement { }
```

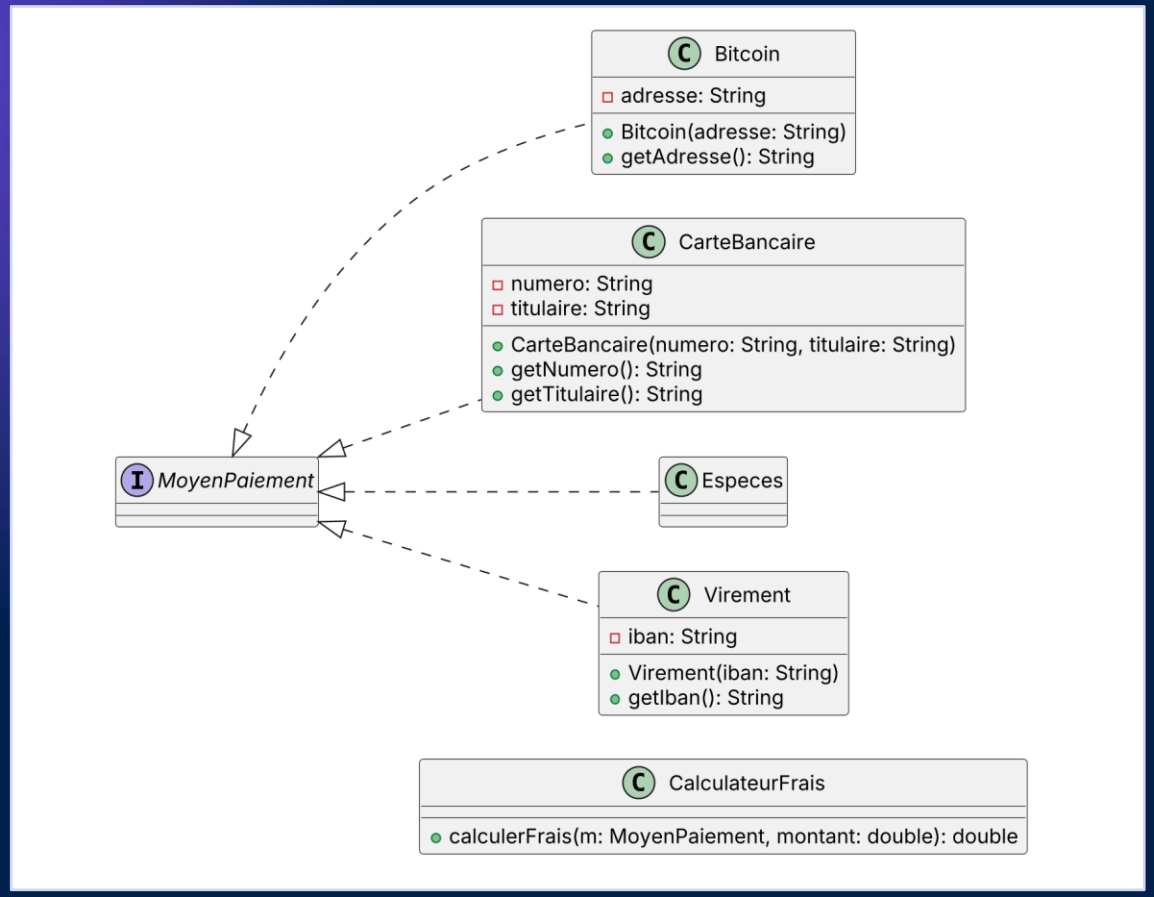
`sealed` : hiérarchie fermée

`record` : données immuables

= type de données algébrique (ADT), comme en Kotlin, Scala ou Rust : le compilateur connaît toutes les variantes possibles.

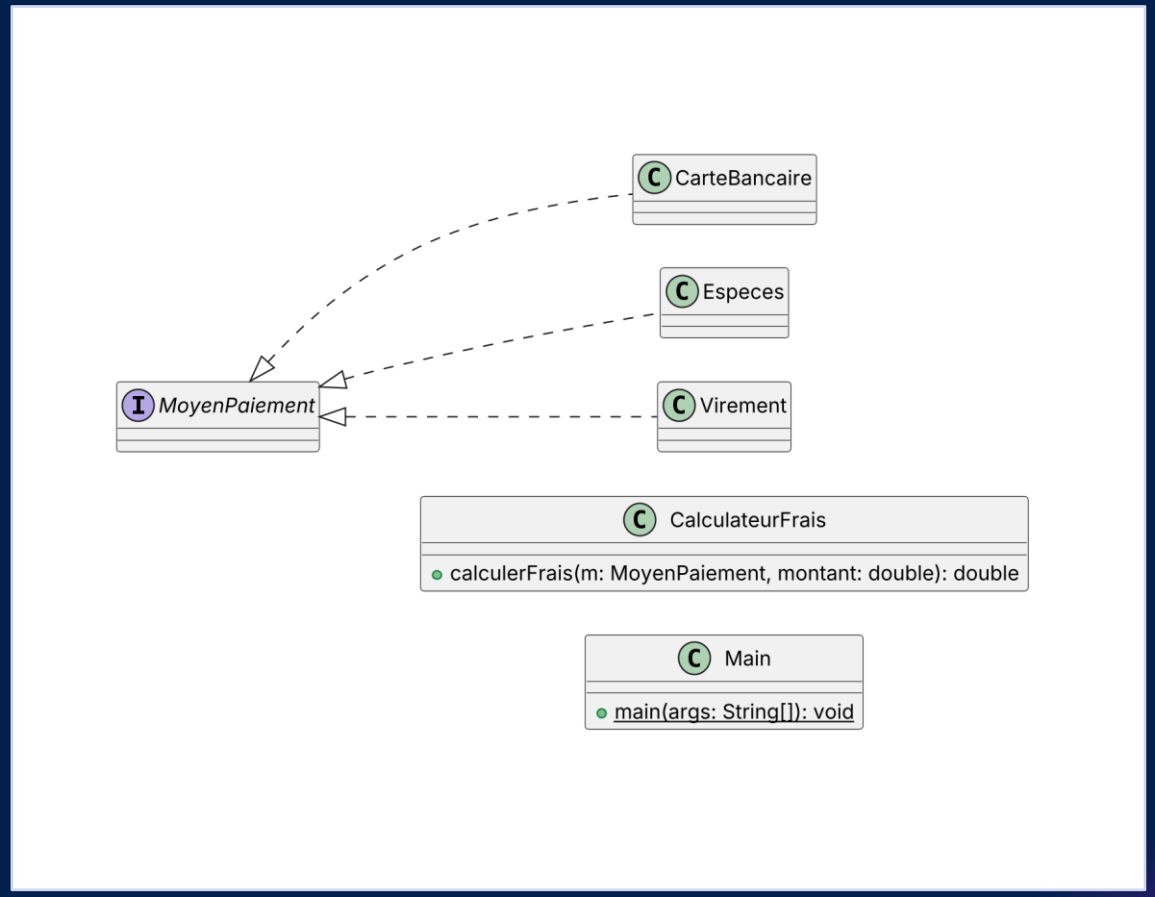
DIAGRAMMES DE CLASSES : AVANT / APRÈS

Avant : hiérarchie ouverte



Interface ordinaire : Bitcoin peut s'ajouter n'importe où.
instanceof : un cas oublié devient un bug à l'exécution.

Après : hiérarchie scellée



sealed + permits : la liste des sous-types est fermée.
switch exhaustif : un oubli = erreur de compilation.

02

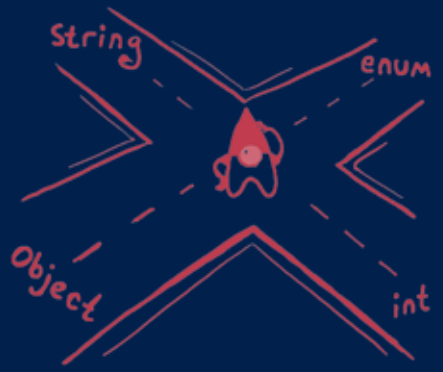
LE PATTERN MATCHING



de instanceof à switch · Java 16 à 21

LE PATTERN MATCHING, C'EST QUOI ?

- **Vérifier qu'une valeur a une certaine forme** et, dans le même geste, en récupérer les éléments
- Trois ingrédients, avec notre paiement :
  - **la valeur :** `paiement`
  - **le motif :** `CarteBancaire cb`
  - **le test :** correspond ? oui, on continue ; non, cas suivant
  - **la variable liée :** `cb`, déjà typée et utilisable
- Idée venue de Haskell, Scala, Rust ; intégrée à Java par le projet Amber



LE PATTERN MATCHING EN JAVA : L'ÉVOLUTION

Version	Date de sortie	Fonctionnalités	Pourquoi
Avant Java 13	avant 2020	instanceof suivi d'un cast manuel	Aucun moyen de lier une variable au test des types
Java 14	mars 2020	switch expressions ; instanceof avec motif	Préparer le terrain : le switch devient donc une expression
Java 16	mars 2021	instanceof avec motif	Fusionner le test de type et le cast
Java 17 à 20	2021 à 2023	switch avec motifs (4 préversions)	Affiner la conception grâce aux retours des développeurs
Java 21 (LTS)	sept. 2023	switch avec motifs, record patterns	Exploiter une hiérarchie scellée, exhaustivité vérifiée

INSTANCEOF : FINI LE CAST REDONDANT

Avant Java 16

```
if (paiement instanceof CarteBancaire) {
    CarteBancaire cb =
        (CarteBancaire) paiement;
    System.out.println(cb.numero());
}
```

Depuis Java 16

```
if (paiement instanceof CarteBancaire cb) {
    System.out.println(cb.numero());
}
```



SWITCH ET PATTERN MATCHING (JAVA 21)

switch permet donc de tester des types

```
String description = switch (paiement) {
    case CarteBancaire cb -> "Carte " + cb.numero();
    case Virement v       -> "Virement " + v.iban();
    case Especes e        -> "Espèces";
};
```

Plus de cascade de if / else if instanceof avec des casts partout.

Garde : when

```
case CarteBancaire cb
    when cb.numero().startsWith("4")
        -> "Carte Visa";
```

Une condition supplémentaire pour affiner un case.

Gérer null : case null

```
case null -> "Aucun paiement";
```

Sans lui, un switch sur null lève une NullPointerException.

LE COMBO : SWITCH EXHAUSTIF

Aujourd'hui : aucun default nécessaire

```
double frais = switch (m) {
    case CarteBancaire cb -> 1.5;
    case Virement v       -> 0.0;
    case Especes e        -> 0.0;
    // aucun default !
};
```

Le compilateur vérifie lui-même que les trois case couvrent tout le domaine : c'est un **switch exhaustif**.

Demain : on ajoute Bitcoin à la liste permits

```
sealed interface MoyenPaiement
    permits CarteBancaire, Virement,
            Especes, Bitcoin { }
```

↓

```
error: the switch expression does
not cover all possible input values
```

Cela permet désormais, que si un cas est oublié, cela ne deviant pas un bug “non-détécté”, car le compilateur l’affiche directement comme une erreur

EXEMPLE : SEALED, RECORD, SWITCH


```java
public sealed interface MoyenPaiement
        permits CarteBancaire, Virement, Especes { }

public record CarteBancaire(String numero, String titulaire) implements MoyenPaiement { }
public record Virement(String iban)                          implements MoyenPaiement { }
public record Especes()                                      implements MoyenPaiement { }

public class CalculateurFrais {
    public double calculerFrais(MoyenPaiement m, double montant) {
        return switch (m) {
            case CarteBancaire cb -> montant * 0.015;
            case Virement v       -> 0.0;
            case Especes e        -> 0.0;
        };  // pas de default : le compilateur sait que tout est couvert
    }
}
```

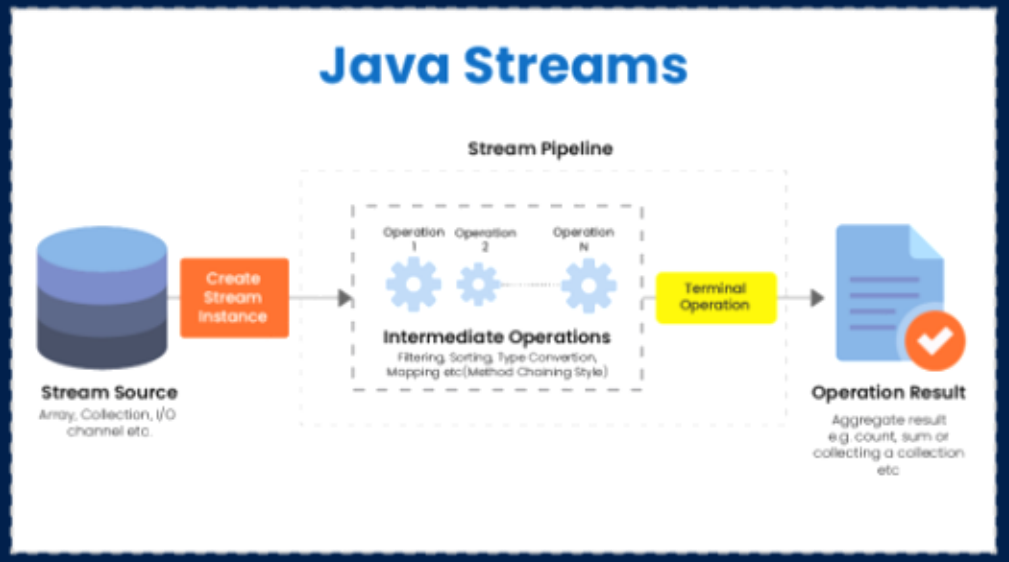
sealed : interface fermée **record** : données immuables **switch** : dispatch exhaustif

LES BÉNÉFICES DE CETTE APPROCHE

- **Sûreté :** un cas oublié = erreur de compilation
- **Concision :** plus de cascade de if / else if, plus de casts
- **Lisibilité :** une case par variante,
- **Évolutivité :** l'ajout d'une classe à permits, fait que le compilateur désigne les switch à mettre à jour
- **Le compromis :** dans une hiérarchie fermée, ajouter une variante oblige à modifier ses switch

03

LE STYLE FONCTIONNEL AVEC LES STREAMS



LAMBDAS ET RÉFÉRENCES DE MÉTHODE

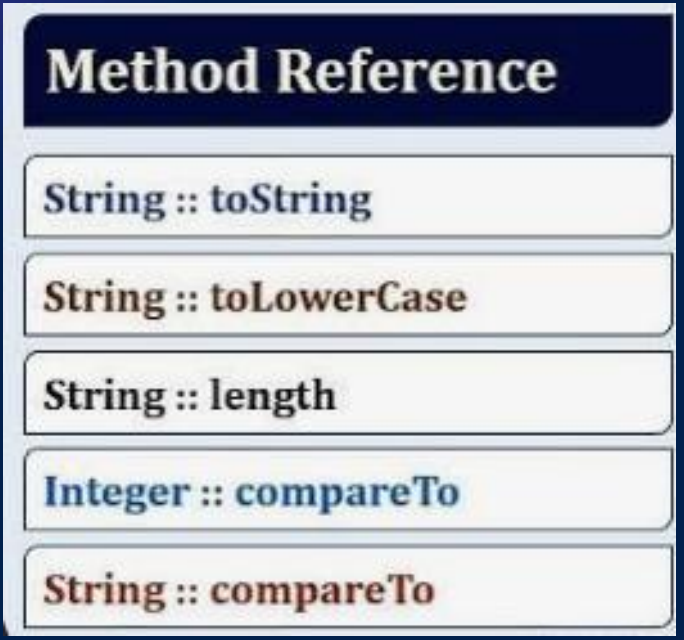
Lambda : une fonction anonyme, écrite avec ->

```
Comparator<String> parLongueur =
    (a, b) -> a.length() - b.length();
```

Référence de méthode : un raccourci avec ::

```
Comparator<String> parLongueur =
    Comparator.comparingInt(String::length);
```

Les quatre formes de référence de méthode



Lambda Expressions	Equivalent Method References
(String s) -> Integer.parseInt(s)	Integer::parseInt
(String s) -> s.toLowerCase()	String::toLowerCase
(int i) -> System.out.println(i)	System.out::println
(Student s) -> s.getName()	Student::getName
() -> s.getName()	s::getName where *'s'* refers to *Student* object which already exist.
() -> new Student()	Student::new

MÊME RÉSULTAT, DEUX FAÇONS DE L'ÉCRIRE


```java
double total = 0;
for (MoyenPaiement m : paiements) {
    if (m instanceof CarteBancaire cb) {
        total += calculerFrais(cb, 100);
    }
}
```

Comment faire : un compteur, une boucle, un test, une accumulation.

```java
double total = paiements.stream()
    .filter(m -> m instanceof CarteBancaire)
    .mapToDouble(m -> calculerFrais(m, 100))
    .sum();
```

Ce qu'on veut : filtrer, transformer, sommer.

Style déclaratif : on décrit le résultat attendu, pas les étapes pour l'obtenir.

UN STREAM EST PARESSEUX

- **Une source**
  - `collection.stream()`, `Stream.of(...)`, `IntStream.range(0, 10)` : ne stocke rien
- **Des opérations intermédiaires**
  - `filter`, `map`, `sorted`, `distinct` : renvoient un nouveau Stream, rien ne s'exécute
- **Une opération terminale**
  - `collect`, `toList`, `forEach`, `reduce`, `count` : déclenche l'exécution
- **Un Stream ne s’utilise qu'une seule fois :** le réutiliser lève une IllegalStateException

FILTER, MAP, COLLECT : PAS À PAS


```
List<String> ibans = paiements.stream()   // [Carte, Virement, Espèces, Virement]
    .filter(m -> m instanceof Virement)   // [Virement, Virement]
    .map(m -> ((Virement) m).iban())      // ["FR76…", "FR14…"]
    .collect(Collectors.toList());        // List<String> : les deux IBAN
```

Depuis Java 16 : .toList() remplace **.collect(Collectors.toList())**, mais renvoie une liste qui n'est pas mutable/modifiable

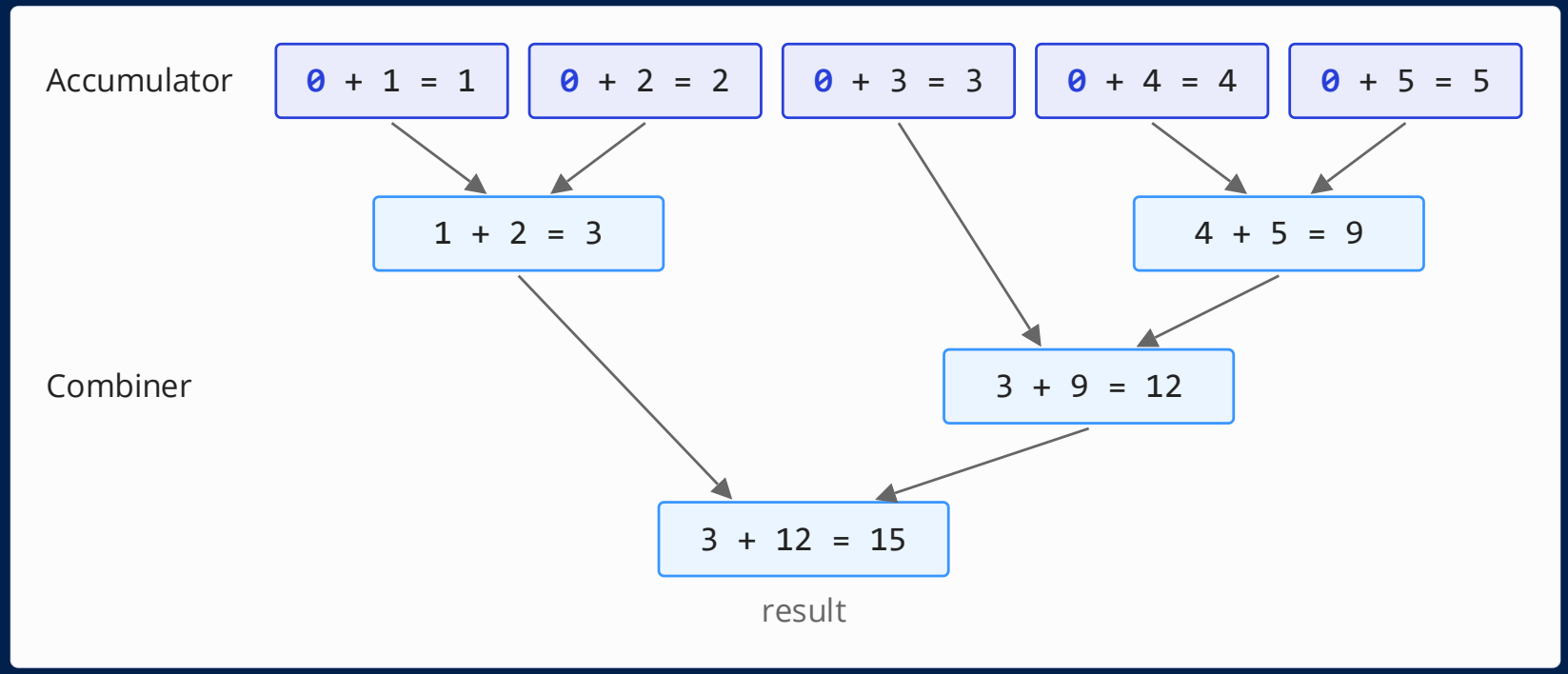
REDUCE : COMBINER EN UNE SEULE VALEUR


```java
int total = List.of(1, 2, 3, 4, 5).stream()
    .reduce(0, Integer::sum);
// total vaut 15
```

`0` : valeur de départ

`(a, b) -> a + b` : combine deux valeurs

`Integer::sum` : même chose, en référence



LES OPÉRATIONS TERMINALES COURANTES

toList()

```
.toList()                          // Java 16
.collect(Collectors.toList())      // Java 8
```

Deux façons de produire une liste (la première est non modifiable).

Collectors.joining

```
String resume = paiements.stream()
    .map(Object::toString)
    .collect(Collectors.joining(", "));
```

Concatène des chaînes avec un séparateur.

average() + orElse()

```
double moyenne = paiements.stream()
    .mapToDouble(m -> calculerFrais(m, 100))
    .average()      // OptionalDouble
    .orElse(0.0);   // si le flux est vide
```

La moyenne d'un flux vide n'existe pas : orElse donne une valeur par défaut.

forEach()

```
paiements.forEach(System.out::println);
```

TOUT ENSEMBLE : SEALED, SWITCH, STREAM


```java
public String recap(List<MoyenPaiement> paiements) {
    return paiements.stream()
        .map(m -> switch (m) {
            case CarteBancaire cb -> "Carte " + cb.numero();
            case Virement v       -> "Virement " + v.iban();
            case Especes e        -> "Espèces";
        })
        .collect(Collectors.joining(", "));
}
```

- **`sealed`** : le switch est exhaustif, sans default
- **pattern matching** : un switch concis, sans cast
- **Stream** : map + collect, sans boucle ni compteur

LIVE-CODING

QCM INTERACTIF

BIBLIOGRAPHIE ET WEBOGRAPHIE

- **Sealed Classes :**
  - OpenJDK, https://openjdk.org/jeps/409
  - https://www.baeldung.com/java-sealed-classes-interfaces
  - https://www.happycoders.eu/java/sealed-classes/

- **Pattern Matching pour instanceof :** OpenJDK, https://openjdk.org/jeps/394
- **Pattern Matching pour switch :** OpenJDK, https://openjdk.org/jeps/441
- **Package java.util.stream:**
  - Documentation Oracle, https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html
  - https://www.happycoders.eu/java/java-stream-reduce/
  - https://fr.scribd.com/document/698602111/Stream
  - https://javatechonline.com/method-reference-in-java-8/